

# Producer–Consumer Using Counting Semaphores and POSIX Threads

## Question

**Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.**

## Main concepts

```
pthread_create()  
pthread_join()  
semaphore  
sem_init()  
sem_wait()  
sem_post()  
producer  
consumer
```

```
saikrishna__90156@DESKTOP-2JF9H5T:~$ nano Practical12.c
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc Practical12.c -o Practical12
saikrishna__90156@DESKTOP-2JF9H5T:~$ ./Practical12
Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Consumer consumed: 2
Producer produced: 3
Consumer consumed: 3
Producer produced: 4
Producer produced: 5
Consumer consumed: 4
Producer produced: 6
Consumer consumed: 5
Producer produced: 7
Producer produced: 8
Consumer consumed: 6
Producer produced: 9
Consumer consumed: 7
Producer produced: 10
Producer produced: 11
Consumer consumed: 8
Producer produced: 12
Consumer consumed: 9
Producer produced: 13
Producer produced: 14
Consumer consumed: 10
Producer produced: 15
Consumer consumed: 11
Producer produced: 16
Consumer consumed: 12
Producer produced: 17
Consumer consumed: 13
Producer produced: 18
Consumer consumed: 14
Producer produced: 19
Consumer consumed: 15
Producer produced: 20
Consumer consumed: 16
```

```
Consumer consumed: 19
Consumer consumed: 20
```

Producer-Consumer completed successfully.

```
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat Practical12.c
```

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 20

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;

        printf("Producer produced: %d\n", i);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
    }
}
```

```

        usleep(100000);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];

        printf("Consumer consumed: %d\n", item);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);

        usleep(150000);
    }

    return NULL;
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

```

```

saikrishna__90156@DESKTOP- X + v

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread,
                  NULL,
                  producer,
                  NULL);

    pthread_create(&consumer_thread,
                  NULL,
                  consumer,
                  NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);

    pthread_mutex_destroy(&mutex);

    printf("\nProducer-Consumer completed successfully.\n");

    return 0;
}
saikrishna__90156@DESKTOP-2JF9H5T:~$ |

```